Introduction

In this lab, you will write and test code on the ESP32 that controls the rotational speed of an attached brushless motor. The motor is controlled by two signals: a PWM speed control and a direction control. You will use the LEDC peripheral to control the PWM signal and the PCNT peripheral to read the encoder signal. Additionally, you will use FreeRTOS Operating System to manage the reading of the motor state and computing the new PWM duty cycle.

Quadrature Encoder

Our motors include a quadrature encoder to determine the motor's speed and direction. Quadrature encoders send two pulse signals, where the pulse frequency encodes the motor rotational speed and the relative phase of the two signals determines the motor direction. Converting the pulse frequency to the motor speed depends on the number of pulses per revolution and the gear ratio. Our motors send 100 pulses per rotation and lack a gearing system, so the rotational speed in revolutions per minute (`rpm`) is computed as a function of pulse frequency (`f`) as:

```
rpm = (60*100)/f
```

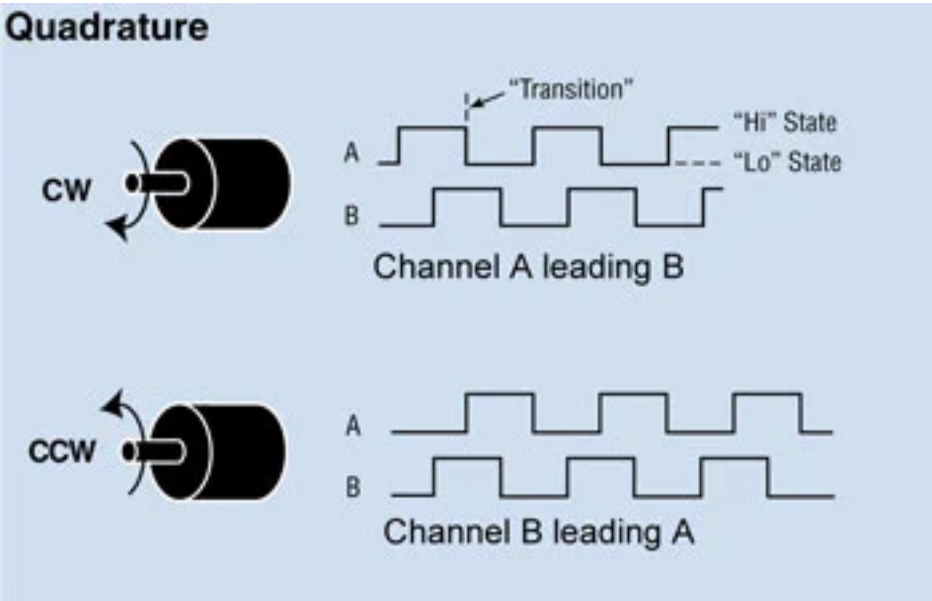

Figure 1: quadrature encoder

(Source: https://www.dynapar.com/technology/encoder_basics/quadrature_encoder/)

Copyright 2025 Jason D. Bakos and Alex Anderson-McLeod

Since we'll only use one direction, you may only use one encoder signal for the motor.

PCNT Peripheral

The PCNT peripheral keeps track of the number of pulses received on an input pin. In this lab we'll use the ESP32 driver for this peripheral. To use it, include the necessary header file:

```
#include "driver/pulse_cnt.h"
```

There are four variables to declare to initialize the PCNT peripheral. Two are used as handles for the PCNT *unit* and PCNT *channel*, since each unit on the chip contains multiple channels, where each channel can track pulses on one signal at a time.

```
pcnt_unit_handle_t mypcnt = 0;
pcnt_channel_handle_t mychannel = 0;
```

You also need to initialize a configuration structure for the unit and the channel:

```
pcnt_unit_config_t myunitconfig = {
  .low_limit = -1,
  .high_limit = (1<<14),
  .intr_priority = 0,
  .flags = {
    .accum_count=0
  }
};

pcnt_chan_config_t mychannelconfig = {
  .edge_gpio_num = <pin>,
  .level_gpio_num = -1,
  .flags = {
    .invert_edge_input = 0,
    .invert_level_input = 0,
    .virt_edge_io_level= 0,
    .io_loop_back = 0
  }
};
```

The value `pcnt_unit_config_t::high_limit` sets the upper limit for the pulse counter, and `1<<14` is the largest possible value for this, which we recommend.

The value `pcnt_chan_config_t::edge_gpio_num` defines which pin is to be monitored for pulses.

Refer to the PCB design for the A/B encoder inputs from motor connectors 1, 2, and 3.

To initialize the PCNT peripheral, use the following sequence:

```
pcnt_new_unit(&myunitconfig,&mypcnt);
pcnt_new_channel(mypcnt,&mychannelconfig,&mychannel);
pcnt_channel_set_edge_action(mychannel,
    PCNT_CHANNEL_EDGE_ACTION_INCREASE, PCNT_CHANNEL_EDGE_ACTION_HOLD);
pcnt_unit_clear_count(mypcnt);
pcnt_unit_enable(mypcnt);
pcnt_unit_start(mypcnt);
```

The `pcnt_channel_set_edge_action` function defines what happens on a positive and negative edge if the signal connected to the PCNT channel. In this case, we'll increment the count on a rising edge and hold on a falling edge. Alternatively, you can count on the rising and falling edge using:

```
pcnt_channel_set_edge_action(mychannel,
    PCNT_CHANNEL_EDGE_ACTION_INCREASE, PCNT_CHANNEL_EDGE_ACTION_INCREASE
    );
```

This will increase the number of "counts" per revolution to 200, providing finer monitoring but requiring more frequent reads of the pulse count to avoid saturation, which in this case would occur after 16384/200 = ~82 revolutions.

The read and subsequently clear the value of the counter, use:

```
pcnt_unit_get_count(mypcnt, &val);
pcnt_unit_clear_count(mypcnt);
```

Note that you must read the count before it reaches `2^14-1 = 16383` counts, otherwise it will hold at that value.

See the link below for full documentation of the PCNT driver:

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/pcnt.html

FreeRTOS

The ESP32 runs FreeRTOS, which allows you to create tasks. Note that the `loop()` function is running in its own FreeRTOS task at the lowest priority. In this lab, you will create two new tasks; one for monitoring the motor speed and one for controlling the PWM signal going to the motor. You will use the `loop()` task to monitor the utilization of the CPU.

To use FreeRTOS, add the following includes:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
```

To create a new task, call the `xTaskCreate()` function, for example:

```
TaskHandle_t mytask_handle;
xTaskCreate(mytask,"my task",1<<16,0,configMAX_PRIORITIES - 1,&
    mytask_handle);
```

In this call, `mytask` is the name of your task function, `"my task"` is a description of the task, `1<<16` is the stack size of the task (64 KB), `0` is a NULL pointer to the arguments passed into the task, `configMAX_PRIORITIES - 1` is the task priority (in this case, the highest priority), and `&mytask_handle` is used to set the handle to the task.

Your task function can be setup as a periodic task using the example below:

```
void mytask (void *args) {
  uint64_t start_task;
  TickType_t xLastWakeTime = xTaskGetTickCount();
  TickType_t xTimeIncrement = pdMS_TO_TICKS(TASK_PERIOD_IN_MS);

  for (;;) {
    start_task = esp_timer_get_time();

    // code here

    taskATotal += esp_timer_get_time() - start_task;
    vTaskDelayUntil(&xLastWakeTime,xTimeIncrement);
  }
}
```

This task function runs in an infinite loop, but the `vTaskDelayUntil()` function forces it to awaken according to a fixed period defined by `TASK_PERIOD_IN_MS ms`. Additionally, it keeps track of its total execution time in a global variable called `taskATotal`, which we can use to measure its CPU utilization.

If you use `start_time = esp_timer_get_time();` in your `setup()` function, you can record the time at which the system initiates execution. This way, the CPU utilization of your task can be obtained by `taskATotal / (esp_timer_get_time()- start_time)`. Note that `esp_timer_get_time()` is functionally the same as `micros()` but potentially more accurate.

More information on FreeRTOS can be obtained at the link below: https://www.freertos.org/Documentation/00-Overview

Controlling the Motor

You can choose any of the three motor connectors to connect your motor. To control the motor, use the LEDC code from the previous lab to control the motor control pins. Also, drive the direction control pin to 0 and set the brake control to 0 (refer to the PCB design for the pin numbers). You can use `digitalWrite()` for these assignments.

Our goal is to force the motor to rotate at a given RPM value, as given in your code. To do this, use a PID controller in its own FreeRTOS task to make adjustments to the PWM value. This task will work together with your motor speed monitoring task to control the motor.

Motor Speed Monitoring Task

Invoke a task to monitor the speed of the motor via PCNT. The speed can be calculated as the value read from PCNT (i.e. `val` above) divided by the period of the monitoring task.

Motor Control Task

Invoke another task to set the LEDC PWM output value to control the motor. In this case, assume

```
error = setpoint - motor_speed;
derivative_error = error - error_read_in_previous_period;
integral_error = error + integral_error
```

You can then multiply each of these by the `Kd, Kd, and Ki` parameters and use the result to set the PWM value.

Monitoring Utilization

Monitor the CPU utilization of both tasks and report each every second on the console using the `loop()` function.

Submitting Your Code

Submit your code using Blackboard.

Copyright 2025 Jason D. Bakos and Alex Anderson-McLeod

Rubric

- 20 points: Use of PCNT to read motor speed
- 20 points: Use of LEDC for motor control
- 20 points: Setup of motor monitoring task
- 20 points: Setup of motor control task
- 20 points: Control of motor

Copyright 2025 Jason D. Bakos and Alex Anderson-McLeod